

Ejercicios de JOINS

Tabla informativa sobre SELECT

Funcionalidad	¿Para qué sirve?	Ejemplo
ORDER BY	Ordena los resultados de una consulta de forma ascendente o descendente.	SELECT * FROM Books ORDER BY Name ASC;
LIMIT	Limita la cantidad de filas que devuelve una consulta.	SELECT * FROM Books LIMIT 3;
GROUP BY	Agrupar filas que tienen valores iguales. Se usa comúnmente con funciones como COUNT, SUM o AVG.	SELECT Author, COUNT(*) FROM Books GROUP BY Author;
INNER JOIN	Devuelve solo los registros que tienen coincidencias en ambas tablas.	SELECT * FROM Books INNER JOIN Authors ON Books.Author = Authors.ID;
LEFT JOIN	Devuelve todos los registros de la tabla izquierda y las coincidencias de la tabla derecha. Si no hay coincidencia, muestra NULL.	SELECT * FROM Books LEFT JOIN Authors ON Books.Author = Authors.ID;
CROSS JOIN	Combina cada fila de una tabla con cada fila de otra tabla.	SELECT * FROM Books CROSS JOIN Authors;

Creación de tablas

```
CREATE TABLE Authors (  
    ID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL  
);
```

```
CREATE TABLE Books (  
    ID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL,  
    Author INTEGER,  
    FOREIGN KEY (Author) REFERENCES Authors(ID)  
);
```

```
CREATE TABLE Customers (  
    ID INTEGER PRIMARY KEY,  
    Name TEXT NOT NULL,
```

```

        Email TEXT NOT NULL
    );

CREATE TABLE Rents (
    ID INTEGER PRIMARY KEY,
    BookID INTEGER,
    CustomerID INTEGER,
    State TEXT NOT NULL,
    FOREIGN KEY (BookID) REFERENCES Books(ID),
    FOREIGN KEY (CustomerID) REFERENCES Customers(ID)
);

```

Inserción de datos

```

INSERT INTO Authors (ID, Name) VALUES
(1, 'Miguel de Cervantes'),
(2, 'Dante Alighieri'),
(3, 'Takehiko Inoue'),
(4, 'Akira Toriyama'),
(5, 'Walt Disney');

```

```

INSERT INTO Books (ID, Name, Author) VALUES
(1, 'Don Quijote', 1),
(2, 'La Divina Comedia', 2),
(3, 'Vagabond 1-3', 3),
(4, 'Dragon Ball 1', 4),
(5, 'The Book of the 5 Rings', NULL);

```

```

INSERT INTO Customers (ID, Name, Email) VALUES
(1, 'John Doe', 'j.doe@email.com'),
(2, 'Jane Doe', 'jane@doe.com'),
(3, 'Luke Skywalker', 'darth.son@email.com');

```

```

INSERT INTO Rents (ID, BookID, CustomerID, State) VALUES
(1, 1, 2, 'Returned'),
(2, 2, 2, 'Returned'),
(3, 1, 1, 'On time'),
(4, 3, 1, 'On time'),
(5, 2, 2, 'Overdue');

```

1. Obtener todos los libros y sus autores, en caso de tenerlos

```
SELECT
    Books.ID,
    Books.Name AS Book,
    Authors.Name AS Author
FROM Books
LEFT JOIN Authors
ON Books.Author = Authors.ID;
```

The screenshot shows the DB Browser for SQLite application. The main window displays a SQL query in the editor and its results in a table. The query is a SELECT statement that joins the Books and Authors tables. The results table shows 5 rows of data, including book titles and authors. The bottom panel shows the execution status and a log of the query.

DB Browser for SQLite - /Users/krist/Desktop/join books.db

Database Structure | Browse Data | Edit Pragmas | **Execute SQL**

Mode: Text

1 Don Quijote

Editing row=1, column=1
Type: Text / Numeric; Size: 11 character(s)

Identity Select an identity to connect

DBHub.io Local Current Database

ID	Book	Author
1	Don Quijote	Miguel de Cervantes
2	La Divina Comedia	Dante Alighieri
3	Vagabond 1-3	Takehiko Inoue
4	Dragon Ball 1	Akira Toriyama
5	The Book of the 5 Rings	NULL

Execution finished without errors.
Result: 5 rows returned in 8ms
At line 1:
SELECT
Books.ID,
Books.Name AS Book,
Authors.Name AS Author
FROM Books

SQL Log Plot DB Schema **Remote**

UTF-8

2. Obtener todos los libros que no tienen autor

```
SELECT
    Books.ID,
    Books.Name AS Book
FROM Books
LEFT JOIN Authors
ON Books.Author = Authors.ID
WHERE Authors.ID IS NULL;
```

The screenshot shows the DB Browser for SQLite application. The main window displays the following SQL query:

```
1 SELECT
2   Books.ID,
3   Books.Name AS Book
4 FROM Books
5 LEFT JOIN Authors
6 ON Books.Author = Authors.ID
7 WHERE Authors.ID IS NULL;
```

Below the query editor, a table with one row is displayed:

ID	Book
1	The Book of the 5 Rings

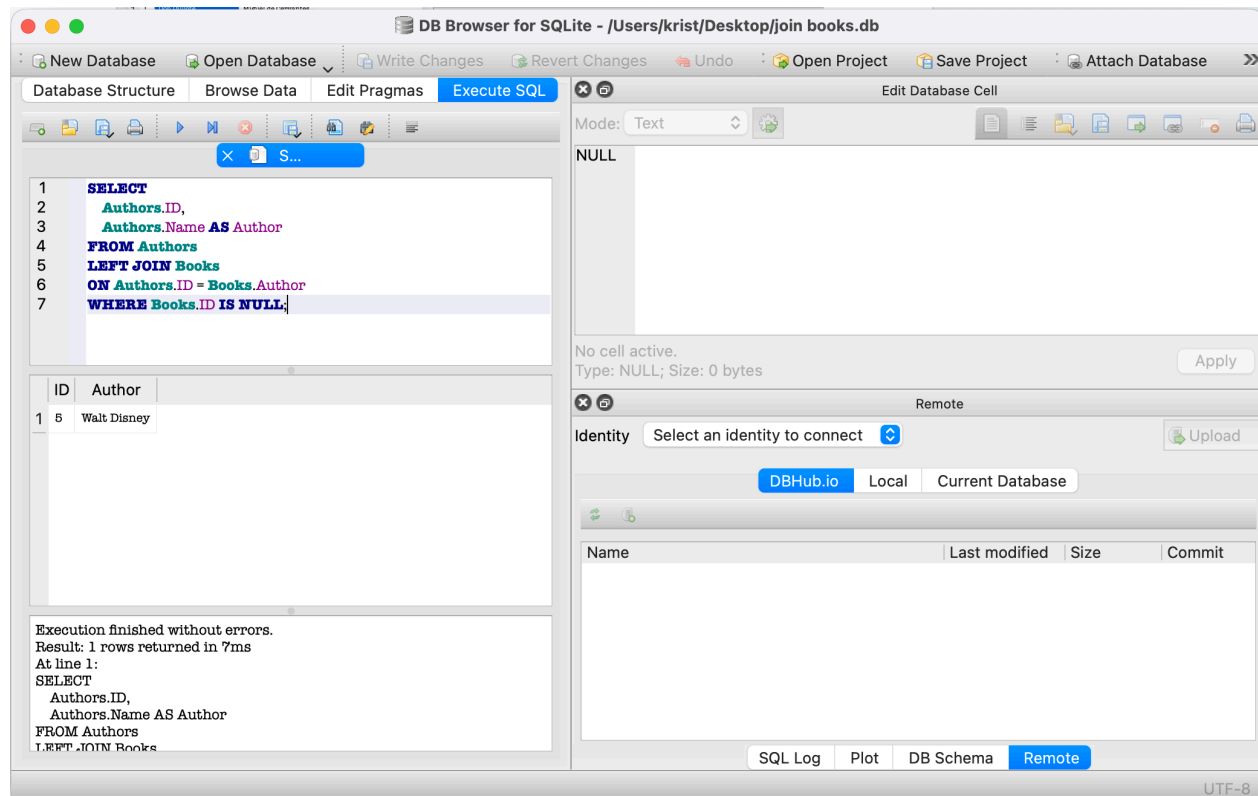
The status bar at the bottom indicates "Execution finished without errors. Result: 1 rows returned in 7ms".

The right sidebar shows the "Edit Database Cell" panel with the text "NULL". Below this, the "Remote" section is visible, showing a dropdown for "Identity" with the text "Select an identity to connect". The "Remote" section also includes buttons for "DBHub.io", "Local", and "Current Database".

The bottom status bar shows "UTF-8".

3. Obtener todos los autores que no tienen libros

```
SELECT
    Authors.ID,
    Authors.Name AS Author
FROM Authors
LEFT JOIN Books
ON Authors.ID = Books.Author
WHERE Books.ID IS NULL;
```



4. Obtener todos los libros que han sido rentados en algún momento

```
SELECT DISTINCT
    Books.ID,
    Books.Name AS Book
FROM Books
INNER JOIN Rents
ON Books.ID = Rents.BookID;
```

The screenshot shows the DB Browser for SQLite interface. The 'Execute SQL' tab is active, displaying the following SQL query:

```
1 SELECT DISTINCT
2   Books.ID,
3   Books.Name AS Book
4 FROM Books
5 INNER JOIN Rents
6 ON Books.ID = Rents.BookID;
```

Below the query editor, a table with 3 rows is displayed:

ID	Book
1	Don Quijote
2	La Divina Comedia
3	Vagabond 1-3

The status bar at the bottom indicates: 'Execution finished without errors. Result: 3 rows returned in 7ms. At line 1: SELECT DISTINCT Books.ID, Books.Name AS Book FROM Books INNER JOIN Rents'.

The right sidebar shows the 'Edit Database Cell' panel with 'Mode: Text' and 'NULL' content. Below it, the 'Remote' connection panel is visible, showing 'Identity: Select an identity to connect' and 'DBHub.io' as the selected connection method.

5. Obtener todos los libros que nunca han sido rentados

```
SELECT
    Books.ID,
    Books.Name AS Book
FROM Books
LEFT JOIN Rents
ON Books.ID = Rents.BookID
WHERE Rents.ID IS NULL;
```

The screenshot shows the DB Browser for SQLite interface. The SQL editor contains the following query:

```
1 SELECT
2   Books.ID,
3   Books.Name AS Book
4 FROM Books
5 LEFT JOIN Rents
6 ON Books.ID = Rents.BookID
7 WHERE Rents.ID IS NULL;
```

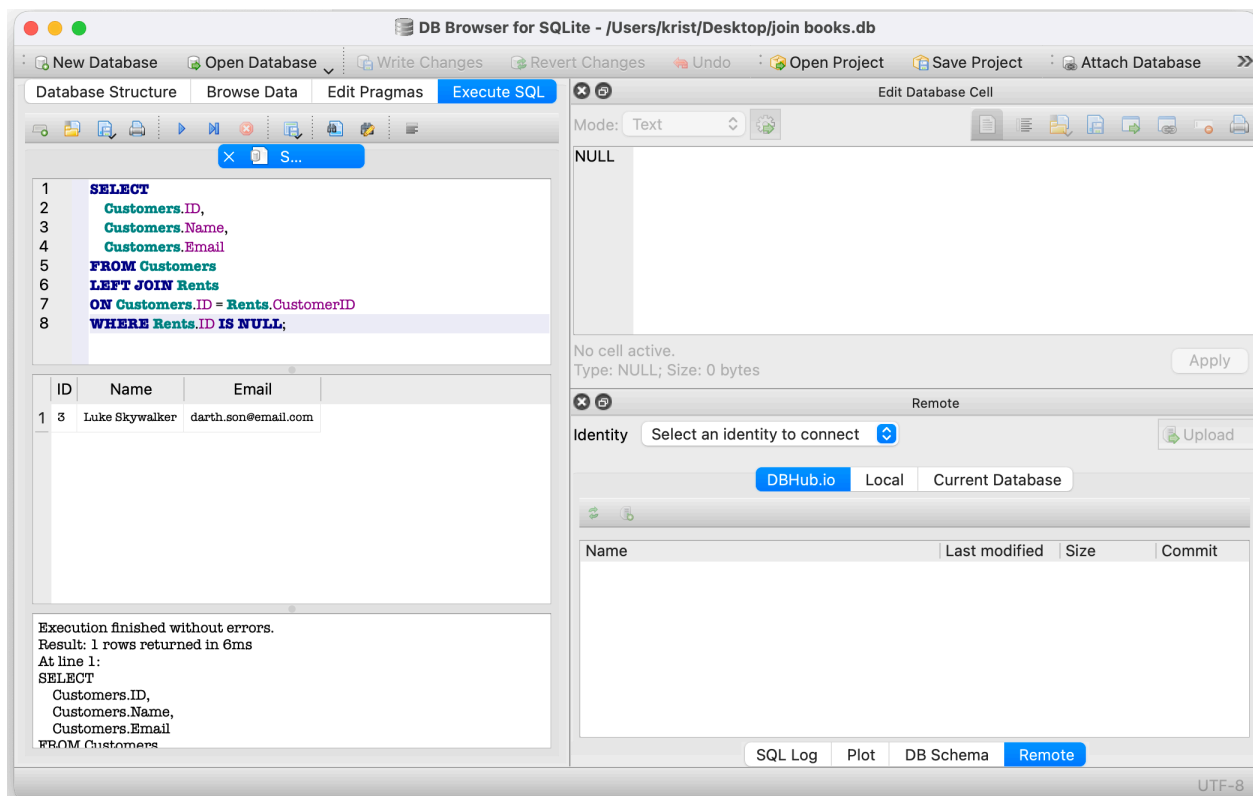
The results pane displays two rows of data:

ID	Book
4	Dragon Ball 1
5	The Book of the 5 Rings

The status bar at the bottom indicates: "Execution finished without errors. Result: 2 rows returned in 8ms. At line 1: SELECT Books.ID, Books.Name AS Book FROM Books LEFT JOIN Rents".

6. Obtener todos los clientes que nunca han rentado un libro

```
SELECT
    Customers.ID,
    Customers.Name,
    Customers.Email
FROM Customers
LEFT JOIN Rents
ON Customers.ID = Rents.CustomerID
WHERE Rents.ID IS NULL;
```



7. Obtener todos los libros que han sido rentados y están en estado “Overdue”

```
SELECT
    Books.ID,
    Books.Name AS Book,
    Rents.State
FROM Books
INNER JOIN Rents
ON Books.ID = Rents.BookID
WHERE Rents.State = 'Overdue';
```

